DBMS – ADVANCED SQL QUERY SYSTEM

Project Title : Lenskart – E-Commerce Order Management System

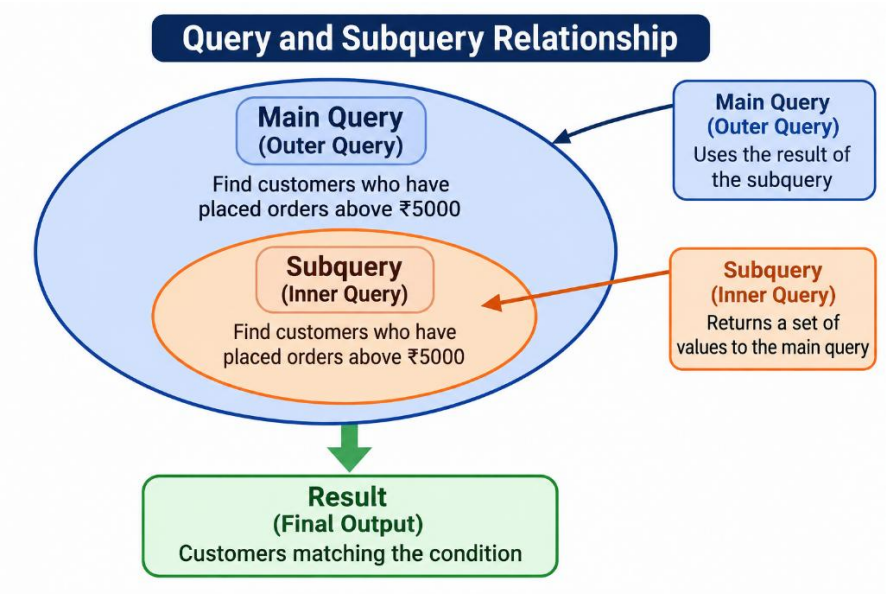
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

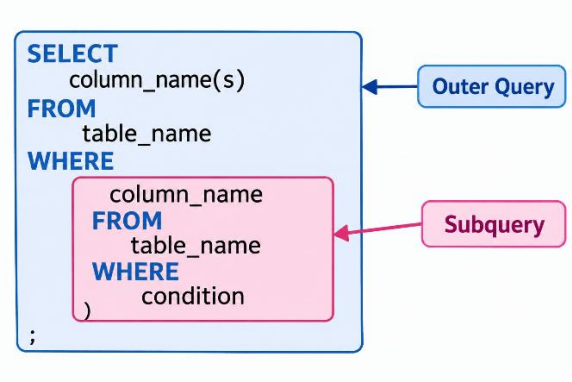
1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

A subquery is a query written inside another SQL query.



Basic Structure



Types of Subqueries

Types of Subqueries

Type	Description
Single-row subquery	Returns one value.
Multiple-row subquery	Returns multiple values.
Nested subquery	A subquery inside another subquery.
Correlated subquery	Inner query depends on the outer query.

Example

Find orders whose amount is greater than the average order amount.

```
1   SELECT Order_ID, Customer_ID, Total_Amount
2   FROM Orders
3   WHERE Total_Amount >
4         (SELECT AVG(Total_Amount)
5          FROM Orders);
6
7   ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
8   ---------- ----------- ------------
9         101           1         2499
10        103           3         2999
11        105           5         3499
12        109           9         2699
13        110          10         3999
14
```

Explanation

The inner query:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

Assuming the Product table contains:

- Product_ID

- Product_Name
- Price

Query

```
SELECT
    Product_ID,
    Product_Name,
    Price
FROM Product
WHERE Price >
     (SELECT AVG(Price)
      FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------
    PRICE
----------
     201
Classic Eyeglasses
     1200

     202
Stylish Sunglasses
     1500
```

Explanation

The inner query calculates the average price:

SELECT AVG(Price) FROM Product;

The outer query displays products whose price is greater than the calculated average.

The actual output depends on the Product data in the database.

2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SELECT
    Product_ID,
    Product_Name,
    Price
FROM Product
WHERE Price <
     (SELECT AVG(Price)
      FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
    PRICE
----------
     203
Reading Glasses
      800

     204
Contact Lenses
     1000
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SELECT
    Customer_ID,
    SUM(Total_Amount) AS Total_Purchase
FROM Orders
GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1           2499
          2           1899
          3           2999
          4           1599
          5           3499
          6           1299
          7           2199
          8           1799
          9           2699
         10           3999

10 rows selected.
```

Step 2: Find the Maximum Purchase

```
SELECT MAX(Total_Purchase)
FROM
(
    SELECT
        Customer_ID,
        SUM(Total_Amount) AS Total_Purchase
    FROM Orders
    GROUP BY Customer_ID
);

MAX(TOTAL_PURCHASE)
-------------------
               3999
```

The inner query calculates the total purchase for each customer

The outer query finds the maximum value among those totals.

3.1 Display Customer Details with Maximum Purchase

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY
    c.Customer_ID,
    c.Customer_Name
HAVING SUM(o.Total_Amount) =
(
    SELECT MAX(Total_Purchase)
    FROM
    (
        SELECT
            SUM(Total_Amount) AS Total_Purchase
        FROM Orders
        GROUP BY Customer_ID
    )
);

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_PURCHASE
----------- ---------------------------------------------------- --------------
         10 Harini                                                         3999
```

Explanation

The innermost query:

SELECT SUM(Total_Amount)

FROM Orders

GROUP BY Customer_ID;

calculates the purchase amount for every customer.

The next query:

SELECT MAX(Total_Purchase)

finds the highest purchase amount.

The main query displays the customer whose purchase amount matches that maximum.

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Total_Amount
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
WHERE o.Total_Amount >
(
    SELECT AVG(Total_Amount)
    FROM Orders
)
ORDER BY o.Total_Amount DESC;

CUSTOMER_ID CUSTOMER_NAME                                          ORDER_ID
----------- -------------------------------------------------- ----------
TOTAL_AMOUNT
------------
         10 Harini                                                    110
       3999

          5 Vignesh                                                   105
       3499

          3 Karthik                                                   103
       2999
```

Purpose

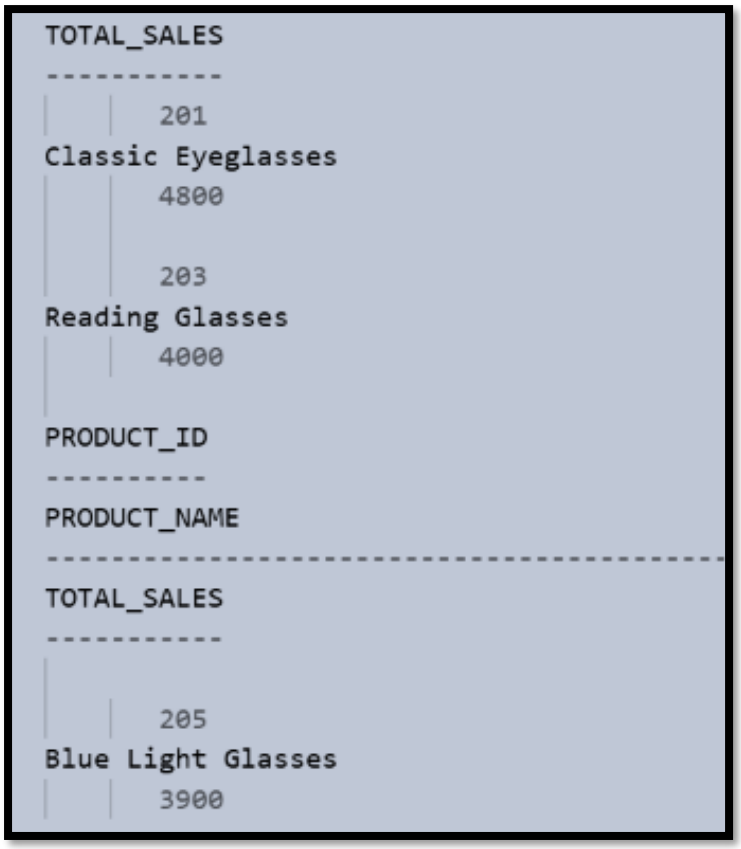
This report identifies customers who placed orders whose value is greater than the overall average order value.

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SELECT
    p.Product_ID,
    p.Product_Name,
    SUM(oi.Quantity * oi.Price) AS Total_Sales
FROM Product p
JOIN Order_Item oi
    ON p.Product_ID = oi.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
HAVING SUM(oi.Quantity * oi.Price) >
(
    SELECT AVG(Product_Sales)
    FROM
    (
        SELECT
            SUM(Quantity * Price) AS Product_Sales
        FROM Order_Item
        GROUP BY Product_ID
    )
)
ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_SALES
-----------
```



Purpose

This query identifies products whose total sales are greater than the average sales generated by the products.

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Total_Orders,
    SUM(o.Total_Amount) AS Total_Purchase,
    AVG(o.Total_Amount) AS Average_Order_Value
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY
    c.Customer_ID,
    c.Customer_Name
ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_ORDERS
----------- ---------------------------------------------------- ------------
TOTAL_PURCHASE AVERAGE_ORDER_VALUE
-------------- -------------------
         10 Harini                                                          1
          3999                3999

          5 Vignesh                                                        1
          3499                3499

          3 Karthik                                                        1
          2999                2999
```

Report Contains

- Customer ID
- Customer Name
- Number of orders
- Total purchase amount
- Average order value

7. Advanced Product Sales Report

This report combines product and order details.

```
SELECT
    p.Product_ID,
    p.Product_Name,
    COUNT(oi.Order_ID) AS Number_of_Orders,
    SUM(oi.Quantity) AS Total_Quantity_Sold,
    SUM(oi.Quantity * oi.Price) AS Total_Sales,
    AVG(oi.Price) AS Average_Selling_Price
FROM Product p
JOIN Order_Item oi
    ON p.Product_ID = oi.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- ------------------- ----------- ---------------------
       201
Classic Eyeglasses
                 2                   4        4800                  1200


       203
Reading Glasses
                 2                   5        4000                   800
```

Purpose

This report helps the business identify:

- Number of orders for each product
- Total quantity sold
- Total sales
- Average selling price
- Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SELECT
    c.Category_ID,
    c.Category_Name,
    COUNT(DISTINCT oi.Order_ID) AS Total_Orders,
    SUM(oi.Quantity) AS Total_Quantity_Sold,
    SUM(oi.Quantity * oi.Price) AS Total_Sales,
    AVG(oi.Price) AS Average_Price
FROM Category c
JOIN Product p
    ON c.Category_ID = p.Category_ID
JOIN Order_Item oi
    ON p.Product_ID = oi.Product_ID
GROUP BY
    c.Category_ID,
    c.Category_Name
ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------ ------------------- ----------- -------------
           1
Eyeglasses
           6                  12       12700          1100


           2
Sunglasses
```